



# BEAUTY OF CLOUD 2.0 REPORT & ARCHITECTURE DIAGRAM TEMPLATE



University of Sri Jayewardenepura  
IEEE Student Branch



IEEE  
COMPUTER  
SOCIETY

University of Sri Jayewardenepura  
Students' Branches & Chapters

- **Team Name:** BackTrack
- **Scenario Chosen:** 2
- **Cloud Platform(s) Used:** GCP

## Team Members

| # | Full Name                     | Role in Team    | University             | Faculty | Email                      | Contact Number |
|---|-------------------------------|-----------------|------------------------|---------|----------------------------|----------------|
| 1 | Inusha Thathsara Gunassekara  | Team Lead       | University of Moratuwa | IT      | Inusha.thathsara@gmail.com | 0786962450     |
| 2 | Madhura Ravishan Abeywickrama | Cloud Architect | University of Moratuwa | IT      | madhuraravishan@gmail.com  | 0702566207     |

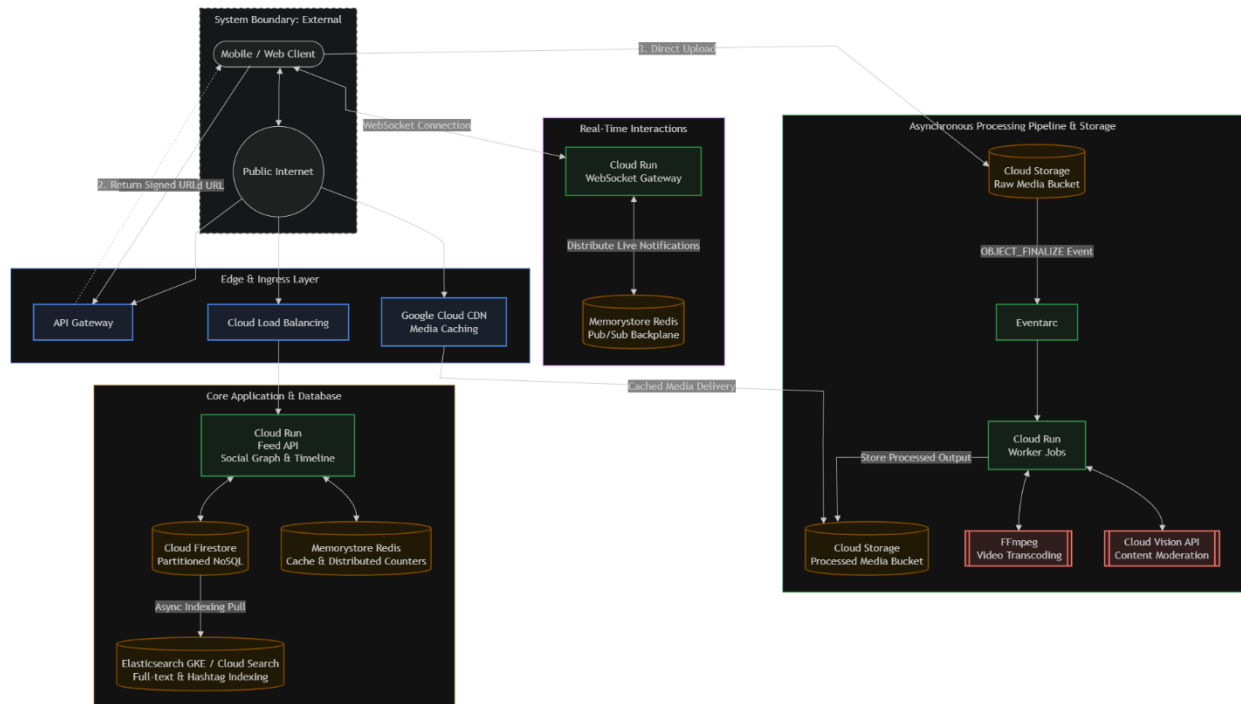
## 1. Section 1 - Problem Understanding

Global social platforms face an extreme computational asymmetry: read-heavy traffic from viewers vastly dwarfs the write traffic generated by content creators. The primary engineering challenge is not simply storing data, but managing massive, unpredictable media payload bottlenecks and the spiraling egress bandwidth costs associated with viral content. Furthermore, real-time interactions (likes, comments, notifications) must arrive promptly without overwhelming primary database locks. Our solution focuses on decoupling media ingestion from the compute backend entirely, solving database write-contention for viral posts, and implementing a highly optimized caching tier to ensure algorithmic feed generation remains fast and scalable.

## 2. Section 2 - Proposed Solution Overview

We propose a highly decoupled, event-driven platform built natively on Google Cloud. Instead of passing media through backend APIs, client devices upload media directly to secure cloud storage. This action triggers serverless background workers that automatically resize images, transcode video for adaptive streaming, and run AI-driven content moderation. To handle the read-heavy load, the system utilizes a hybrid in-memory caching engine (Redis) to deliver personalized feeds in milliseconds, backed by a global Content Delivery Network (CDN) to serve heavy media assets locally, driving down latency and infrastructure costs simultaneously.

## 3. Section 3 - Architecture Diagram



## 4. Section 4 - Technology Choices & Justification

- Direct-to-Cloud Storage & CDN:** To control costs—which are heavily driven by media storage and bandwidth—we bypass application servers for media ingestion. Clients upload via ephemeral Signed URLs directly to Cloud Storage. Cloud CDN serves delivery, preventing 90%+ of traffic from ever hitting the storage origin and drastically reducing egress fees.
- Cloud Run & Redis Pub/Sub Backplane:** Cloud Run scales to zero during off-peak hours and scales horizontally during viral spikes. Because Cloud Run containers are ephemeral, we bind all WebSocket instances to a shared Memorystore (Redis) Pub/Sub channel. This stateful backplane ensures real-time interactions, like comments and DMs, arrive promptly regardless of which container the client is connected to.
- FFmpeg (HLS/DASH) & Cloud Vision API:** To meet the requirement for fast global video delivery with low buffering, our Cloud Run worker uses FFmpeg to transcode videos into HTTP Live Streaming (HLS) and DASH formats, allowing the mobile client to dynamically adapt bitrates based on network conditions. We utilize Cloud Vision API for content moderation to catch harmful content automatically.
- Elasticsearch (Search Indexing):** Cloud Firestore is not optimized for fuzzy full-text search. We pipe post metadata to a dedicated Elasticsearch cluster to explicitly solve the requirement for hashtag, content, and user discovery at scale, decoupling search queries from the primary transactional database.

## 5. Section 5 - Scalability & Performance

Our system is explicitly designed to handle extreme traffic skew without write amplification or database lock contention.

- **Hybrid Feed Fan-Out (Read Scalability):** When a standard user posts, an asynchronous task pushes the post ID into the pre-computed Redis timelines of their followers. However, for high-follower celebrity accounts, pushing to millions of Redis lists causes massive write-lock contention. Instead, celebrity posts are aggregated on-the-fly at read-time and merged with the user's cached timeline.
- **Distributed Counters (Viral Write Scalability):** If a post suddenly goes viral, native Firestore will throttle and crash due to its strict document write limit (roughly 1 write per second). We utilize a Distributed Counter pattern: high-velocity engagements (likes) are incremented atomically in Memorystore (Redis) via INCR commands and flushed to persistent Firestore documents in micro-batches every 5 seconds.

## 6. Section 6 - Security, Privacy & Compliance

- **Authentication & Tenant Isolation:** We utilize OAuth 2.0/OIDC. Direct object access is strictly denied; all buckets are private. Microservices operate on dedicated Identity and Access Management (IAM) Service Accounts (Least Privilege). The Transcoding worker has write access to the Processed Media Bucket but is explicitly denied read access to the Firestore user profile database.
- **Data Encryption & Ephemeral Access:** All data is encrypted at rest using AES-256 (Google-managed encryption keys) and in transit via TLS 1.3. Uploads use single-use Signed URLs with a strict 15-minute Time-To-Live (TTL).
- **Direct Messages (DMs):** DMs are stored in an isolated Firestore collection. Strict security rules enforce that only the explicitly authenticated sender and recipient IDs can query the documents, preventing administrative lateral breaches.

## 7. Section 7 - Cost Estimate

| Component            | Estimated Monthly Cost | Notes                                                                 |
|----------------------|------------------------|-----------------------------------------------------------------------|
| Compute (Cloud Run)  | \$100 - \$180          | Pay-per-use; high concurrency settings keep instance counts low.      |
| Database (Firestore) | \$50 - \$100           | Costs kept minimal by serving 95% of feed reads from the Redis cache. |
| Cache (Memorystore)  | \$120 - \$200          | Fixed cost for a highly available, multi-AZ Redis instance.           |

| Component               | Estimated Monthly Cost | Notes                                                                                                                              |
|-------------------------|------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Storage (Cloud Storage) | \$150 - \$300          | Mitigated by Object Lifecycle Policies: Processed media moves from Standard to Nearline after 30 days, and Archive after 180 days. |
| Delivery (Cloud CDN)    | \$150 - \$300          | Caching edge delivery is vastly cheaper than paying raw origin egress.                                                             |
| Search & ML             | \$100 - \$150          | Elasticsearch node overhead and Cloud Vision API moderation calls.                                                                 |
| Total                   | \$670 - \$1,230        | Cost scales sub-linearly due to aggressive CDN cache hit ratios.                                                                   |

## 8. Section 8 - Availability & Resilience

Given this is a consumer social platform, brief latency degradation is tolerable, but data loss or feed unavailability is not. We prioritize Multi-Availability Zone (Multi-AZ) resilience within a single region over a highly expensive multi-region deployment.

- **Graceful Degradation:** If the Memorystore (Redis) cache cluster fails, the Feed API is designed with a circuit breaker to fall back to querying Firestore directly for a reverse-chronological feed. The feed will load slower, but the platform remains online.
- **Asynchronous Retries:** If the Cloud Vision API or Transcoding worker fails during a traffic spike, the event remains in the Pub/Sub dead-letter queue and retries with exponential backoff, ensuring uploaded media is never permanently lost.

## 9. Section 9 - Monitoring & Observability

To ensure proactive incident response, we focus on specific platform health indicators rather than generic CPU metrics:

- **CDN Cache Hit Ratio:** If this drops below 85%, origin egress costs will spike exponentially. We alert immediately on ratio drops.
- **Dead-Letter Queue (DLQ) Depth:** We monitor Eventarc/Pub/Sub DLQs. A spike indicates the asynchronous media pipeline is failing, meaning users are uploading content but it is not appearing on the platform.
- **P99 API Latency:** Tracked via Cloud Trace to ensure feed generation remains under 200ms even during viral traffic events.

## 10. Section 10 - Conclusion & Future Improvements

While this architecture delivers a highly scalable, decoupled media platform that controls spiraling bandwidth costs, it relies on a heuristic-based timeline merge for feed generation. If we had more time and resources, the immediate next step would be implementing a dedicated machine learning pipeline (using Vertex AI) for personalized feed ranking, decoupled from the core API. Additionally, we would implement predictive pre-fetching on the mobile client downloading the first 5 seconds of the next video in the feed into the local device cache while the user is watching the current video, ensuring absolute zero-buffering transitions.